What is Ensemble Learning?

Ensemble is the art of combining diverse set of learners (individual models) together to improvise on the stability and predictive power of the model. In the above example, the way we combine all the predictions together will be termed as Ensemble Learning.

In this article, we will talk about a few ensemble techniques widely used in the industry. Before we get into techniques, let's first understand how do we actually get different set of learners. Models can be different from each other for a variety of reasons, starting from the population they are built upon to the modeling used for building the model.

Here are the top 4 reasons for a model to be different. They can be different because of a mix of these factors as well:

1. Difference in population

2. Difference in hypothesis

3. Difference in modeling technique

4. Difference in initial seed

The literary meaning of word ‘ensemble’ is *group*. Ensemble methods involve group of predictive models to achieve a better accuracy and model stability. Ensemble methods are known to impart supreme boost to tree based models.

Like every other model, a tree based model also suffers from the plague of bias and variance. Bias means, ‘how much on an average are the predicted values different from the actual value.’ Variance means, ‘how different will the predictions of the model be at the same point if different samples are taken from the same population’.

You build a small tree and you will get a model with low variance and high bias. How do you manage to balance the trade off between bias and variance ?

Normally, as you increase the complexity of your model, you will see a reduction in prediction error due to lower bias in the model. As you continue to make your model more complex, you end up over-fitting your model and your model will start suffering from high variance.

Error in Ensemble Learning (Variance vs. Bias)

The error emerging from any model can be broken down into three components mathematically. Following are these component :

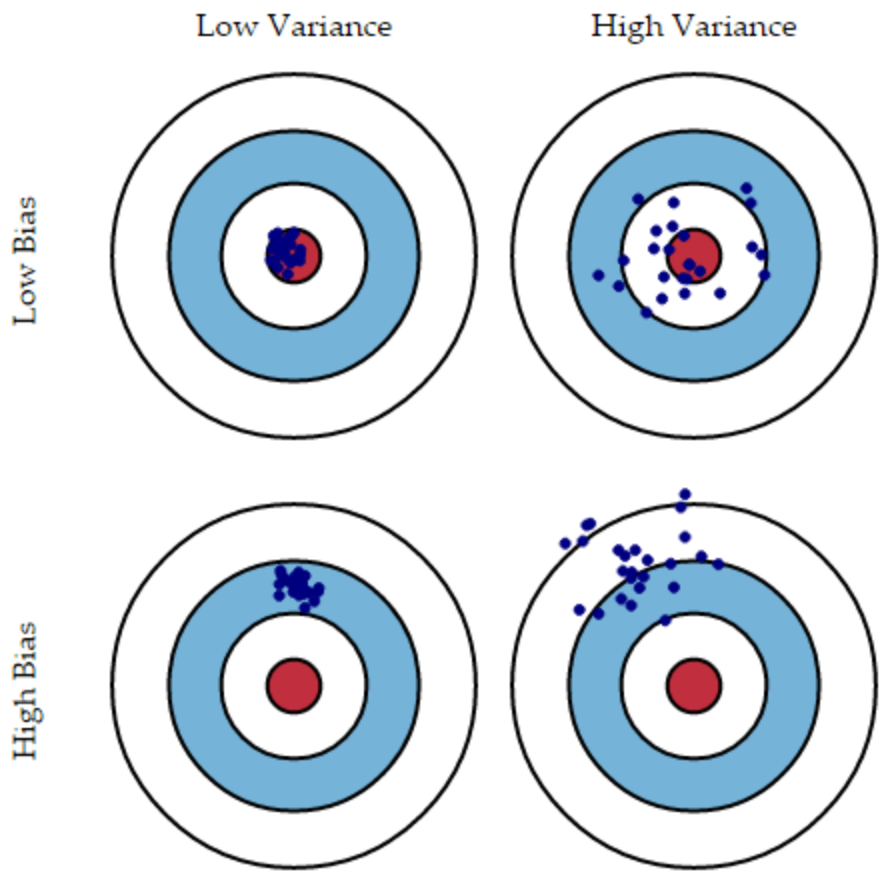
$$Err(x) = \left(E[\hat{f}\,(x)] - f(x)\right)^2 + E\left[\hat{f}\,(x) - E[\hat{f}\,(x)]\right]^2 + \sigma_e^2$$

$$Err(x) = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Why is this important in the current context? To understand what really goes behind an ensemble model, we need to first understand what causes error in the model. We will briefly introduce you to these errors and give an insight to each ensemble learner in this regards.

Bias error is useful to quantify how much on an average are the predicted values different from the actual value. A high bias error means we have a under-performing model which keeps on missing important trends.

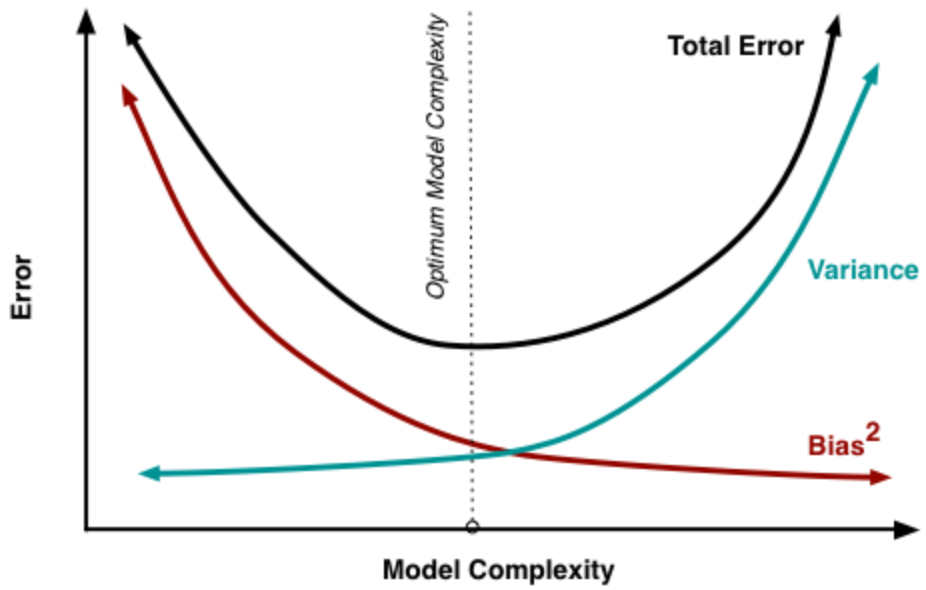
Variance on the other side quantifies how are the prediction made on same observation different from each other. A high variance model will over-fit on your training population and perform badly on any observation beyond training. Following diagram will give you more clarity (Assume that red spot is the real value and blue dots are predictions) :



Credit : Scott Fortman

Normally, as you increase the complexity of your model, you will see a reduction in error due to lower bias in the model. However, this only happens till a particular point. As you continue to make your model more complex, you end up over-fitting your model and hence your model will start suffering from high variance.

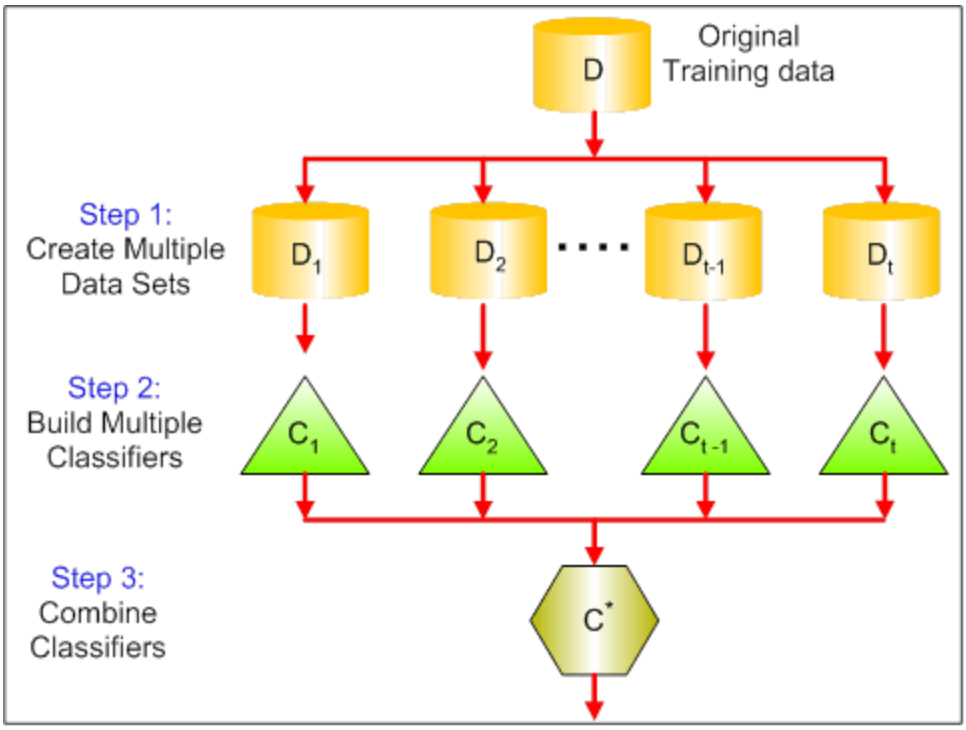
A champion model should maintain a balance between these two types of errors. This is known as the **trade-off management** of bias-variance errors. Ensemble learning is one way to execute this trade off analysis.



Credit : Scott Fortman

Some Commonly used Ensemble learning techniques

1. <u>Bagging</u> : Bagging tries to implement similar learners on small sample populations and then takes a mean of all the predictions. In generalized bagging, you can use different learners on different population. As you can expect this helps us to reduce the variance error.



The steps followed in bagging are:

1. **Create Multiple DataSets**:
    - Sampling is done *with replacement* on the original data and new datasets are formed.
    - The new data sets can have a fraction of the columns as well as rows, which are generally hyper-parameters in a bagging model
    - Taking row and column fractions less than 1 helps in making robust models, less prone to overfitting
2. **Build Multiple Classifiers:**
    - Classifiers are built on each data set.
    - Generally the same classifier is modeled on each data set and predictions are made.
3. **Combine Classifiers:**
    - The predictions of all the classifiers are combined using a mean, median or mode value depending on the problem at hand.
    - The combined values are generally more robust than a single model.

Note that, here the number of models built is not a hyper-parameters. Higher number of models are always better or may give similar performance than lower numbers. It can be theoretically shown that the variance of the combined predictions are reduced to 1/n (n: number of classifiers) of the original variance, under some assumptions.

Bootstrapping

Let's begin by defining bootstrapping. This statistical technique consists in generating samples of size B (called bootstrap samples) from an initial dataset of size N by randomly drawing with replacement B observations.

Illustration of the bootstrapping process.

Under some assumptions, these samples have pretty **good statistical properties**: in first approximation, they can be seen as being drawn both directly from the true underlying (and often unknown) data distribution and independently from each others. So, they can be considered as representative and independent samples of the true data distribution (almost i.i.d. samples). The hypothesis that have to be verified to make this approximation valid are twofold. First, the size N of the initial dataset should be large enough to capture most of the complexity of the underlying distribution so that sampling from the dataset is a good approximation of sampling from the real distribution (**representativity**). Second, the size N of the dataset should be large enough compared to the size B of the bootstrap samples so that samples are not too much correlated (**independence**). Notice that in the following, we will sometimes make reference to these properties (representativity and independence) of bootstrap samples: the reader should always keep in mind that **this is only an approximation**.

How does it work?

In Random Forest, we grow multiple trees as opposed to a single tree in CART model (see comparison between CART and Random Forest here. To classify a new object based on attributes, each tree gives a classification and we say the tree "votes" for that class. The forest chooses the classification having the most votes (over all the trees in the forest) and in case of regression, it takes the average of outputs by different trees.

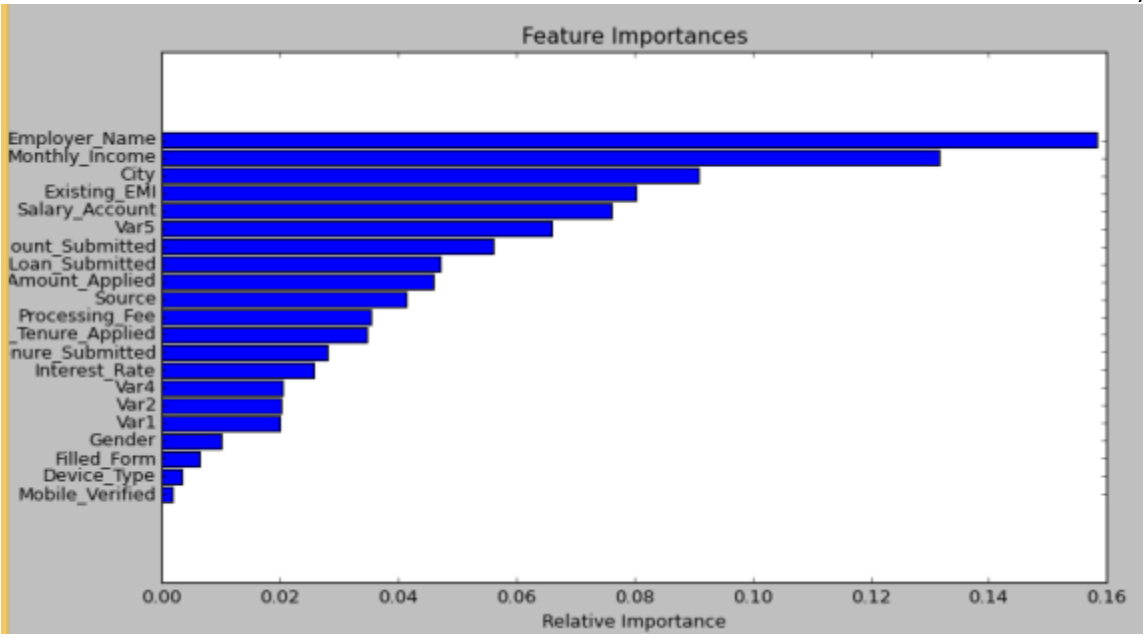
It works in the following manner. Each tree is planted & grown as follows:

1. Assume number of cases in the training set is N. Then, sample of these N cases is taken at random but *with replacement*. This sample will be the training set for growing the tree.
2. If there are M input variables, a number m<M is specified such that at each node, m variables are selected at random out of the M. The best split on these m is used to split the node. The value of m is held constant while we grow the forest.
3. Each tree is grown to the largest extent possible and there is no pruning.
4. Predict new data by aggregating the predictions of the ntree trees (i.e., majority votes for classification, average for regression).

Advantages of Random Forest

- This algorithm can solve both type of problems i.e. classification and regression and does a decent estimation at both fronts.
- One of benefits of Random forest which excites me most is, the power of handle large data set with higher dimensionality. It can handle thousands of input variables and identify most significant variables so it is considered as one of the dimensionality reduction methods. Further, the model outputs **Importance of variable,** which can be a very handy feature (on some random

data set).

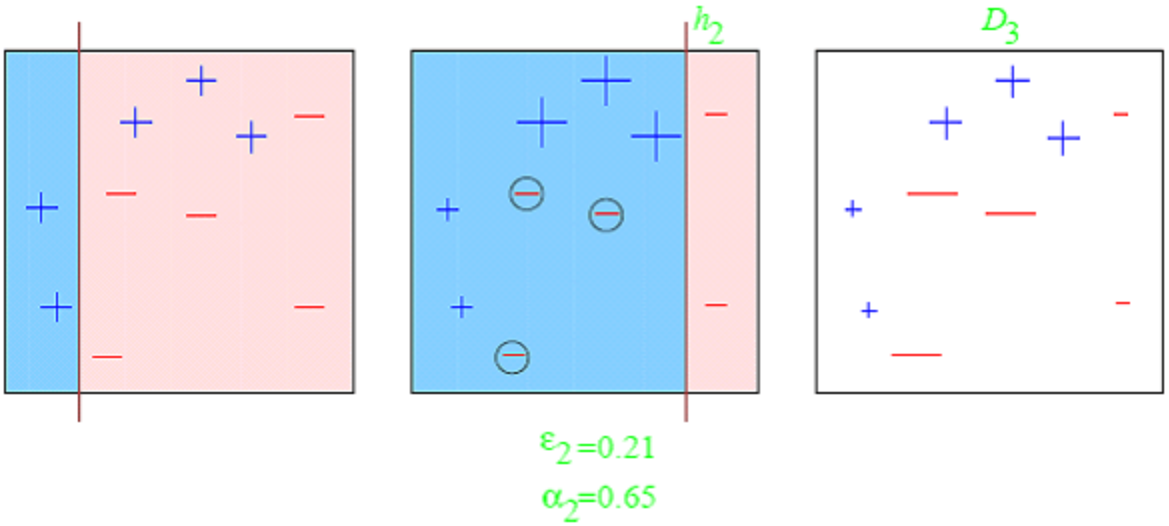


- It has an effective method for estimating missing data and maintains accuracy when a large proportion of the data are missing.
- It has methods for balancing errors in data sets where classes are imbalanced.
- The capabilities of the above can be extended to unlabeled data, leading to unsupervised clustering, data views and outlier detection.
- Random Forest involves sampling of the input data with replacement called as bootstrap sampling. Here one third of the data is not used for training and can be used to testing. These are called the **out of bag** samples. Error estimated on these out of bag samples is known as *out of bag error*. Study of error estimates by Out of bag, gives evidence to show that the out-of-bag estimate is as accurate as using a test set of the same size as the training set. Therefore, using the out-of-bag error estimate removes the need for a set aside test set.

Disadvantages of Random Forest

- It surely does a good job at classification but not as good as for regression problem as it does not give precise continuous nature predictions. In case of regression, it doesn't predict beyond the range in the training data, and that they may over-fit data sets that are particularly noisy.
- Random Forest can feel like a black box approach for statistical modelers – you have very little control on what the model does. You can at best – try different parameters and random seeds!

2. <u>Boosting</u> : Boosting is an iterative technique which adjust the weight of an observation based on the last classification. If an observation was classified incorrectly, it tries to increase the weight of this observation and vice versa. Boosting in general decreases the bias error and builds strong predictive models. However, they may sometimes over fit on the training data.



Observations:

1. **Box 1: Output of First Weak Learner (From the left)**
   - Initially all points have same weight (denoted by their size).
   - The decision boundary predicts 2 +ve and 5 -ve points correctly.
2. **Box 2: Output of Second Weak Learner**
   - The points classified correctly in box 1 are given a lower weight and vice versa.
   - The model focuses on high weight points now and classifies them correctly. But, others are misclassified now.

Similar trend can be seen in box 3 as well. This continues for many iterations. In the end, all models are given a weight depending on their accuracy and a consolidated result is generated.

Definition: The term 'Boosting' refers to a family of algorithms which converts weak learner to strong learners.

Let's understand this definition in detail by solving a problem of spam email identification:

How would you classify an email as SPAM or not? Like everyone else, our initial approach would be to identify 'spam' and 'not spam' emails using following criteria. If:

1. Email has only one image file (promotional image), It's a SPAM
2. Email has only link(s), It's a SPAM
3. Email body consist of sentence like "You won a prize money of $ xxxxxx", It's a SPAM
4. Email from known source, Not a SPAM

Above, we've defined multiple rules to classify an email into 'spam' or 'not spam'. But, do you think these rules individually are strong enough to successfully classify an email? No.

Individually, these rules are not powerful enough to classify an email into 'spam' or 'not spam'. Therefore, these rules are called as **weak learner**.

To convert weak learner to strong learner, we'll combine the prediction of each weak learner using methods like:

- Using average/ weighted average
- Considering prediction has higher vote

For example: Above, we have defined 5 weak learners. Out of these 5, 3 are voted as 'SPAM' and 2 are voted as 'Not a SPAM'. In this case, by default, we'll consider an email as SPAM because we have higher(3) vote for 'SPAM'.

How does it work?

Now we know that, boosting combines weak learner a.k.a. base learner to form a strong rule. An immediate question which should pop in your mind is, '*How boosting identify weak rules?*'

To find weak rule, we apply base learning (ML) algorithms with a different distribution. Each time base learning algorithm is applied, it generates a new weak prediction rule. This is an iterative process. After many iterations, the boosting algorithm combines these weak rules into a single strong prediction rule.

Here's another question which might haunt you, '*How do we choose different distribution for each round?*'

For choosing the right distribution, here are the following steps:

Step 1: The base learner takes all the distributions and assign equal weight or attention to each observation.

Step 2: If there is any prediction error caused by first base learning algorithm, then we pay higher attention to observations having prediction error. Then, we apply the next base learning algorithm.

Step 3: Iterate Step 2 till the limit of base learning algorithm is reached or higher accuracy is achieved.

Finally, it combines the outputs from weak learner and creates a strong learner which eventually improves the prediction power of the model. Boosting pays higher focus on examples which are mis-classified or have higher errors by preceding weak rules.
There are many boosting algorithms which impart additional boost to model's accuracy. In this tutorial, we'll learn about the two most commonly used algorithms i.e. Gradient Boosting (GBM).

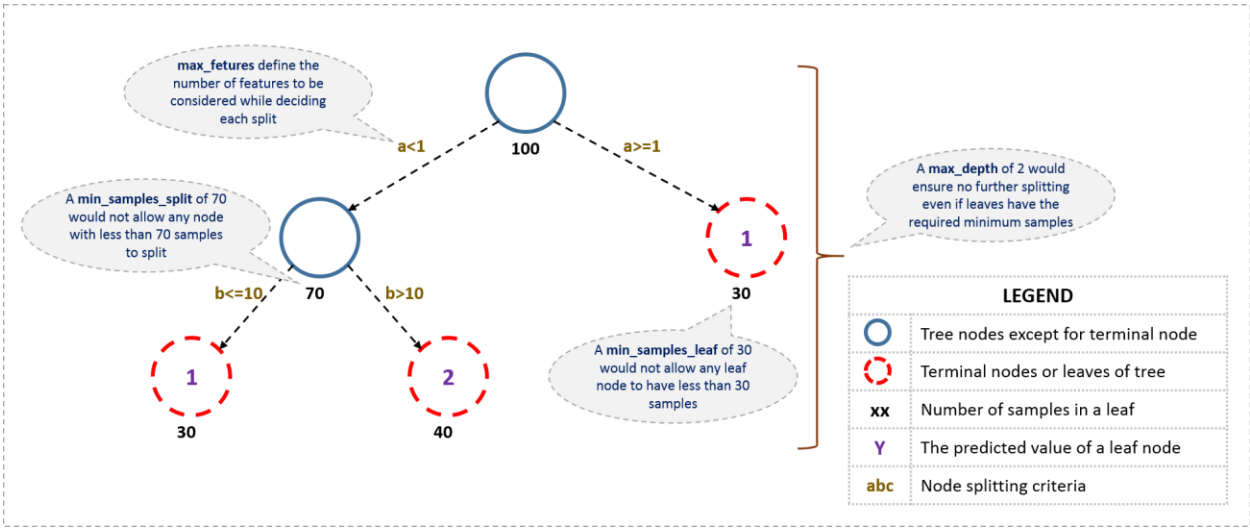
intuition behind gradient boosting algorithm is to repetitively leverage the patterns in residuals and strengthen a model with weak predictions and make it better. Once we reach a stage that residuals do not have any pattern that could be modeled, we can stop modeling residuals (otherwise it might lead to overfitting). Algorithmically, we are minimizing our loss function, such that test loss reach its minima.

GBM Parameters

The overall parameters can be divided into 3 categories:

1. **Tree-Specific Parameters:** These affect each individual tree in the model.
2. **Boosting Parameters:** These affect the boosting operation in the model.
3. **Miscellaneous Parameters:** Other parameters for overall functioning.

I'll start with tree-specific parameters. First, lets look at the general structure of a decision tree:



The parameters used for defining a tree are further explained below. Note that I'm using scikit-learn (python) specific terminologies here which might be different in other software packages like R. But the idea remains the same.

1. **min_samples_split**
    - Defines the minimum number of samples (or observations) which are required in a node to be considered for splitting.
    - Used to control over-fitting. Higher values prevent a model from learning relations which might be highly specific to the particular sample selected for a tree.
    - Too high values can lead to under-fitting hence, it should be tuned using CV.
2. **min_samples_leaf**
    - Defines the minimum samples (or observations) required in a terminal node or leaf.
    - Used to control over-fitting similar to min_samples_split.
    - Generally lower values should be chosen for imbalanced class problems because the regions in which the minority class will be in majority will be very small.
3. **min_weight_fraction_leaf**

    - Similar to min_samples_leaf but defined as a fraction of the total number of observations instead of an integer.
    - Only one of #2 and #3 should be defined.
4. **max_depth**
    - The maximum depth of a tree.
    - Used to control over-fitting as higher depth will allow model to learn relations very specific to a particular sample.
    - Should be tuned using CV.
5. **max_leaf_nodes**
    - The maximum number of terminal nodes or leaves in a tree.
    - Can be defined in place of max_depth. Since binary trees are created, a depth of 'n' would produce a maximum of 2^n leaves.
    - If this is defined, GBM will ignore max_depth.
6. **max_features**
    - The number of features to consider while searching for a best split. These will be randomly selected.
    - As a thumb-rule, square root of the total number of features works great but we should check upto 30-40% of the total number of features.
    - Higher values can lead to over-fitting but depends on case to case.

This is an extremely simplified (probably naive) explanation of GBM's working. The parameters which we have considered so far will affect step 2.2, i.e. model building. Lets consider another set of parameters for managing boosting:

1. **learning_rate**
    - This determines the impact of each tree on the final outcome (step 2.4). GBM works by starting with an initial estimate which is updated using the output of each tree. The learning parameter controls the magnitude of this change in the estimates.
    - Lower values are generally preferred as they make the model robust to the specific characteristics of tree and thus allowing it to generalize well.
    - Lower values would require higher number of trees to model all the relations and will be computationally expensive.
2. **n_estimators**
    - The number of sequential trees to be modeled (step 2)
    - Though GBM is fairly robust at higher number of trees but it can still overfit at a point. Hence, this should be tuned using CV for a particular learning rate.
3. **subsample**
    - The fraction of observations to be selected for each tree. Selection is done by random sampling.
    - Values slightly less than 1 make the model robust by reducing the variance.
    - Typical values ~0.8 generally work fine but can be fine-tuned further.

Apart from these, there are certain miscellaneous parameters which affect overall functionality:

1. **loss**
    - It refers to the loss function to be minimized in each split.

- It can have various values for classification and regression case. Generally the default values work fine. Other values should be chosen only if you understand their impact on the model.

2. **init**
   - This affects initialization of the output.
   - This can be used if we have made another model whose outcome is to be used as the initial estimates for GBM.
3. **random_state**
   - The random number seed so that same random numbers are generated every time.
   - This is important for parameter tuning. If we don't fix the random number, then we'll have different outcomes for subsequent runs on the same parameters and it becomes difficult to compare models.
   - It can potentially result in overfitting to a particular random sample selected. We can try running models for different random samples, which is computationally expensive and generally not used.
4. **verbose**
   - The type of output to be printed when the model fits. The different values can be:
     - 0: no output generated (default)
     - 1: output generated for trees in certain intervals
     - >1: output generated for all trees
5. **warm_start**
   - This parameter has an interesting application and can help a lot if used judicially.
   - Using this, we can fit additional trees on previous fits of a model. It can save a lot of time and you should explore this option for advanced applications
6. **presort**
   - Select whether to presort data for faster splits.
   - It makes the selection automatically by default but it can be changed if needed.

Which is more powerful: GBM or Xgboost?

I've always admired the boosting capabilities that xgboost algorithm. At times, I've found that it provides better result compared to GBM implementation, but at times you might find that the gains are just marginal. When I explored more about its performance and science behind its high accuracy, I discovered many advantages of Xgboost over GBM:

1. **Regularization:**
   - Standard GBM implementation has no regularization like XGBoost, therefore it also helps to reduce overfitting.
   - In fact, XGBoost is also known as '**regularized boosting**' technique.
2. **Parallel Processing:**
   - XGBoost implements parallel processing and is **blazingly faster** as compared to GBM.
   - But hang on, we know that boosting is sequential process so how can it be parallelized? We know that each tree can be built only after the previous one, so what stops us from making a tree using all cores? I hope you get where I'm coming from. Check this link out to explore further.

- XGBoost also supports implementation on Hadoop.

3. **High Flexibility**
   - XGBoost allow users to define **custom optimization objectives and evaluation criteria**.
   - This adds a whole new dimension to the model and there is no limit to what we can do.
4. **Handling Missing Values**
   - XGBoost has an in-built routine to handle missing values.
   - User is required to supply a different value than other observations and pass that as a parameter. XGBoost tries different things as it encounters a missing value on each node and learns which path to take for missing values in future.
5. **Tree Pruning:**
   - A GBM would stop splitting a node when it encounters a negative loss in the split. Thus it is more of a **greedy algorithm**.
   - XGBoost on the other hand make **splits upto the max_depth** specified and then start **pruning** the tree backwards and remove splits beyond which there is no positive gain.
   - Another advantage is that sometimes a split of negative loss say -2 may be followed by a split of positive loss +10. GBM would stop as it encounters -2. But XGBoost will go deeper and it will see a combined effect of +8 of the split and keep both.
6. **Built-in Cross-Validation**
   - XGBoost allows user to run a **cross-validation at each iteration** of the boosting process and thus it is easy to get the exact optimum number of boosting iterations in a single run.
   - This is unlike GBM where we have to run a grid-search and only a limited values can be tested.
7. **Continue on Existing Model**
   - User can start training an XGBoost model from its last iteration of previous run. This can be of significant advantage in certain specific applications.
   - GBM implementation of sklearn also has this feature so they are even on this point.

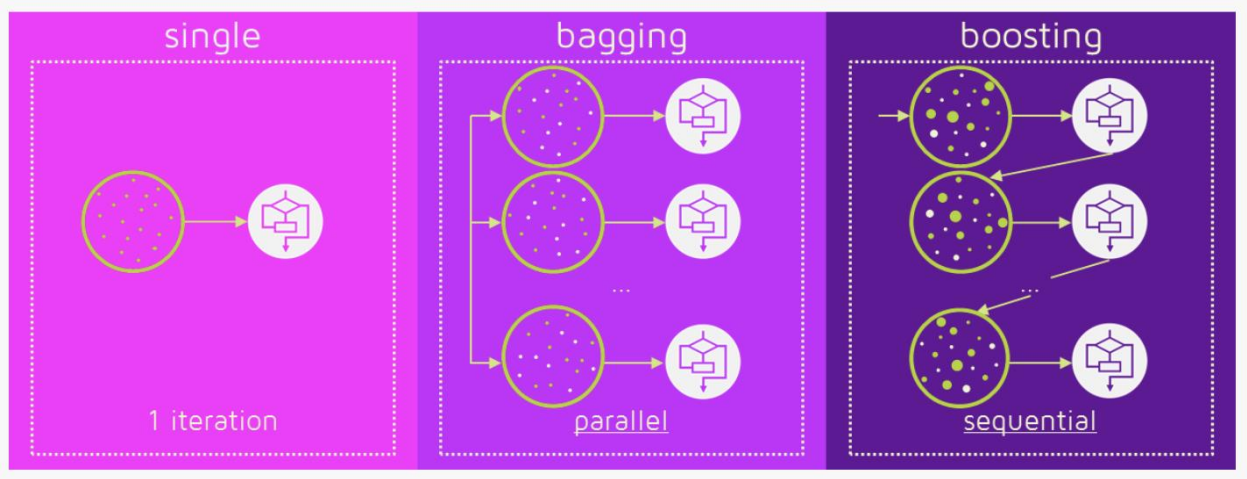
3. Stacking

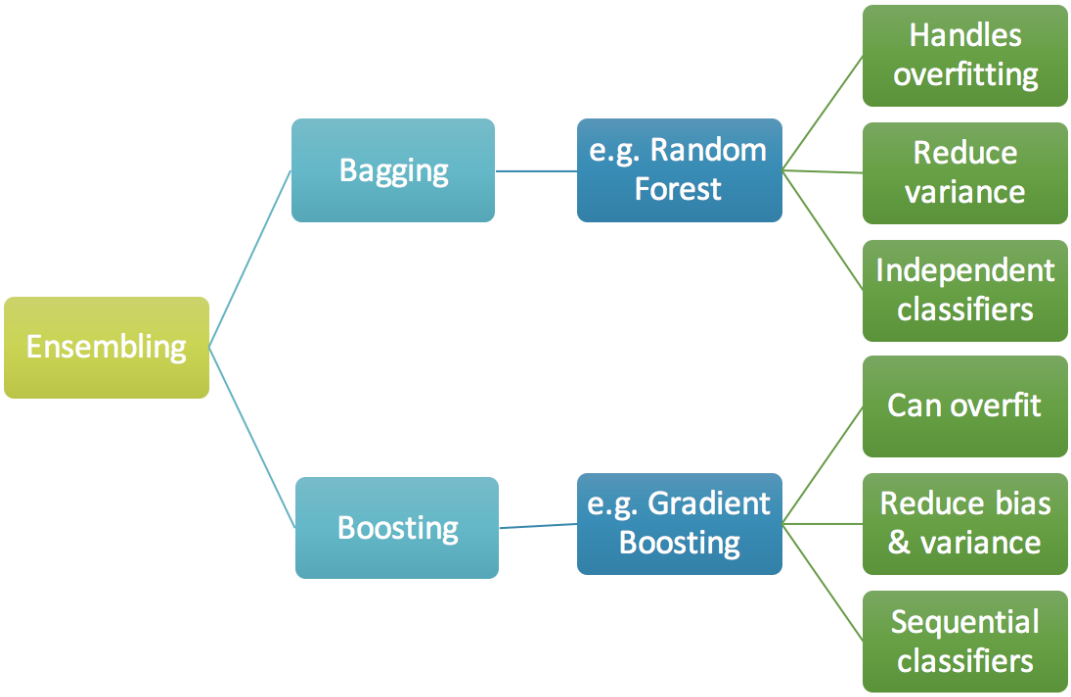
Stacking mainly differ from bagging and boosting on two points. First stacking often considers **heterogeneous weak learners** (different learning algorithms are combined) whereas bagging and boosting consider mainly homogeneous weak learners. Second, stacking learns to combine the base models using a meta-model whereas bagging and boosting combine weak learners following deterministic algorithms.

As we already mentioned, the idea of stacking is to learn several different weak learners and **combine them by training a meta-model** to output predictions based on the multiple predictions returned by these weak models. So, we need to define two things in order to build our stacking model: the L learners we want to fit and the meta-model that combines them.

For example, for a classification problem, we can choose as weak learners a KNN classifier, a logistic regression and a SVM, and decide to learn a neural network as meta-model. Then, the neural network

will take as inputs the outputs of our three weak learners and will learn to return final predictions based on it.





Steps to fit a Gradient Boosting model

Let's consider simulated data as shown in scatter plot below with 1 input (x) and 1 output (y) variables.

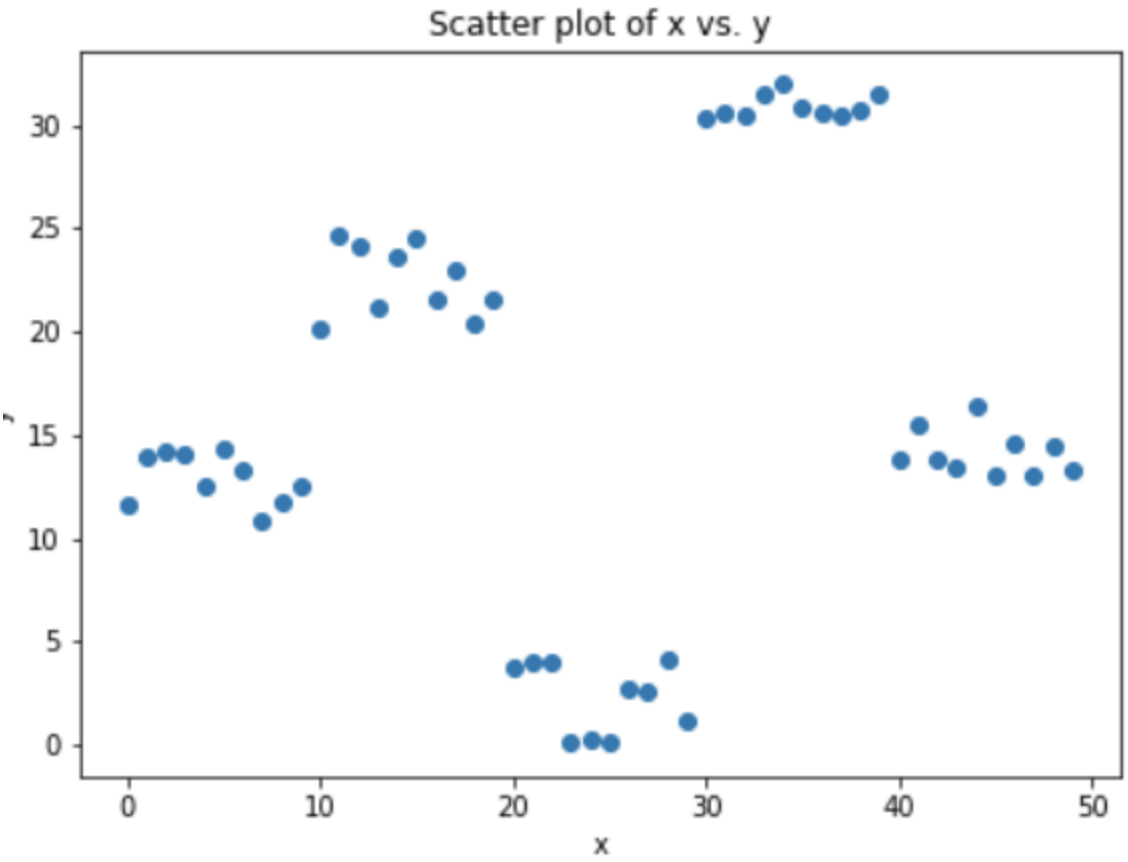


Fig 4. Simulated data (x: input, y: output)

Data for above shown plot is generated using below python code:
Code Chunk 1. Data simulation

1. *Fit a simple linear regressor or decision tree on data (I have chosen decision tree in my code)* ***[call x as input and y as output]***
Code Chunk 2. (Step 1)Using decision tree to find best split (here depth of our tree is 1)

2. *Calculate error residuals. Actual target value, minus predicted target value* ***[e1= y - y_predicted1]***

3. *Fit a new model on error residuals as target variable with same input variables* ***[call it e1_predicted]***

4. *Add the predicted residuals to the previous predictions*
[y_predicted2 = y_predicted1 + e1_predicted]

5. *Fit another model on residuals that is still left. i.e.* ***[e2 = y - y_predicted2]*** *and repeat steps 2 to 5 until it starts overfitting or the sum of residuals become constant. Overfitting can be controlled by consistently checking accuracy on validation data.*
Code Chunk 3. (Steps 2 to 5) Calculate residuals and update new target variable and new predictions

Visualization of working Gradient Boosting Tree

Blue dots (left) plots are input (x) vs. output (y) • Red line (left) shows values predicted by decision tree • Green dots (right) shows residuals vs. input (x) for ith iteration • Iteration represent sequential order of fitting gradient boosting tree

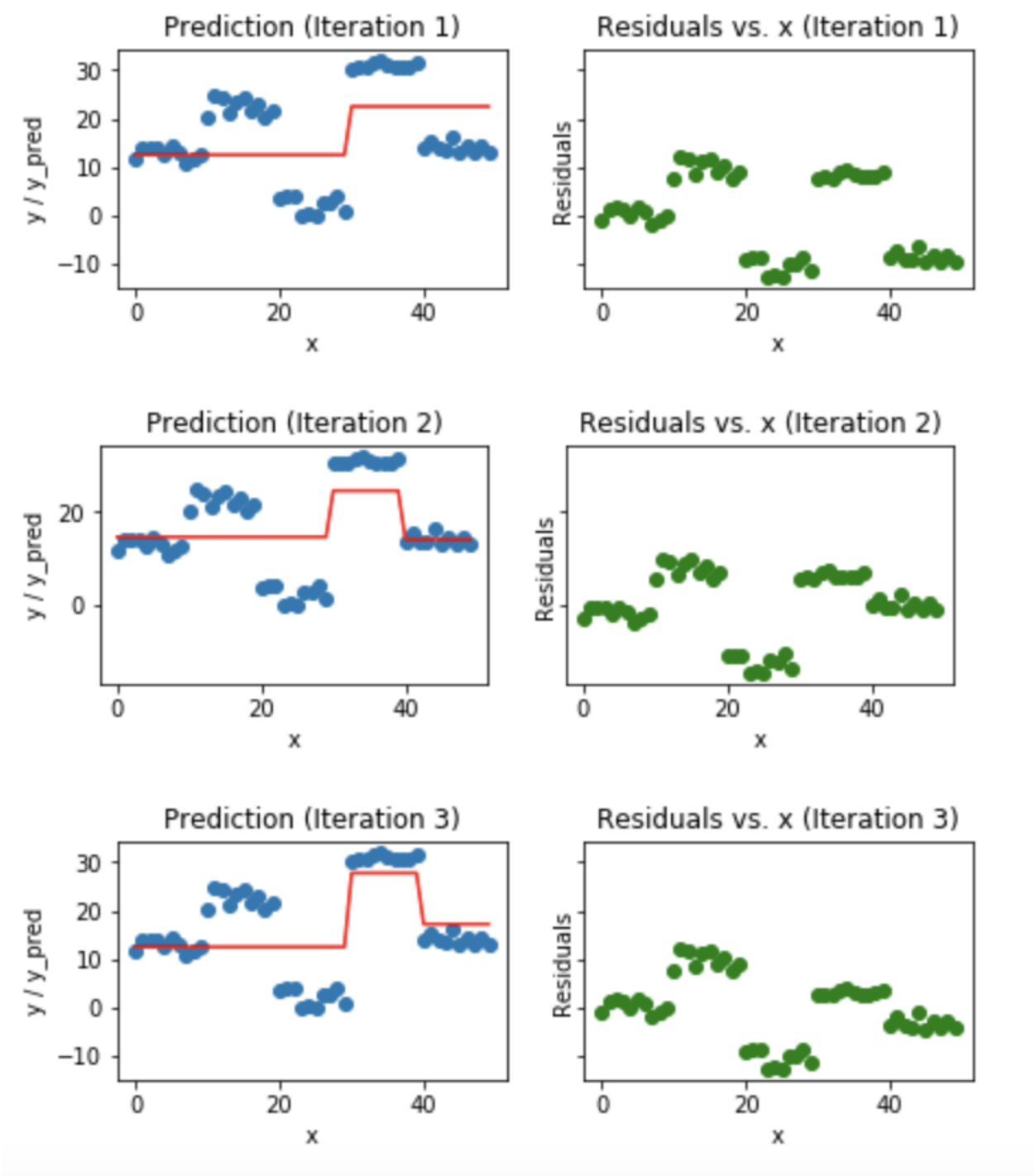


Fig 5. Visualization of gradient boosting predictions (First 4 iterations)

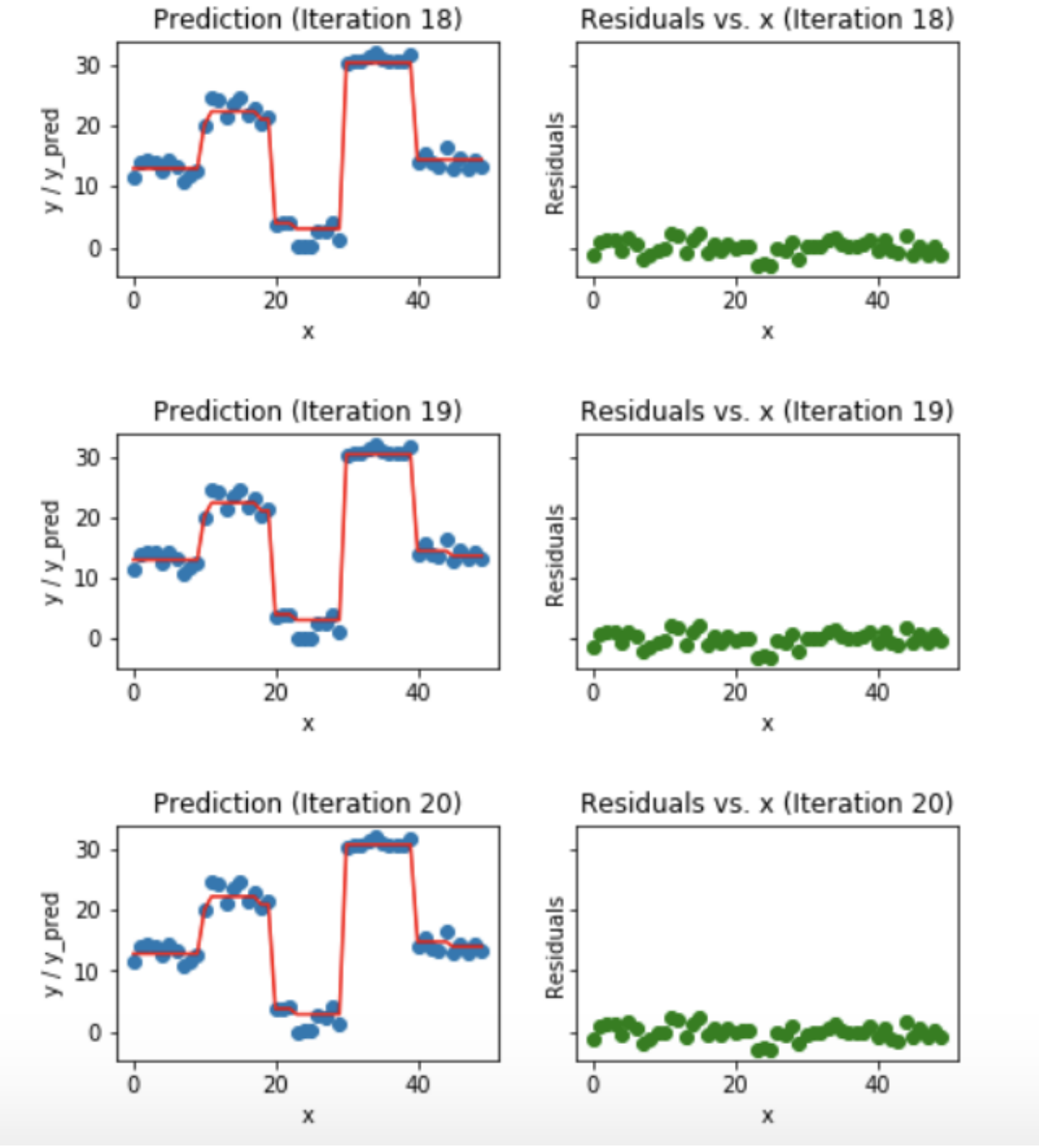


Fig 6. Visualization of gradient boosting predictions (18th to 20th iterations)

We observe that after 20th iteration , residuals are randomly distributed (I am not saying random normal here) around 0 and our predictions are very close to true values. (*iterations* are called *n_estimators* in sklearn implementation). This would be a good point to stop or our model will start overfitting.

Let’s see how our model look like for 50th iteration.

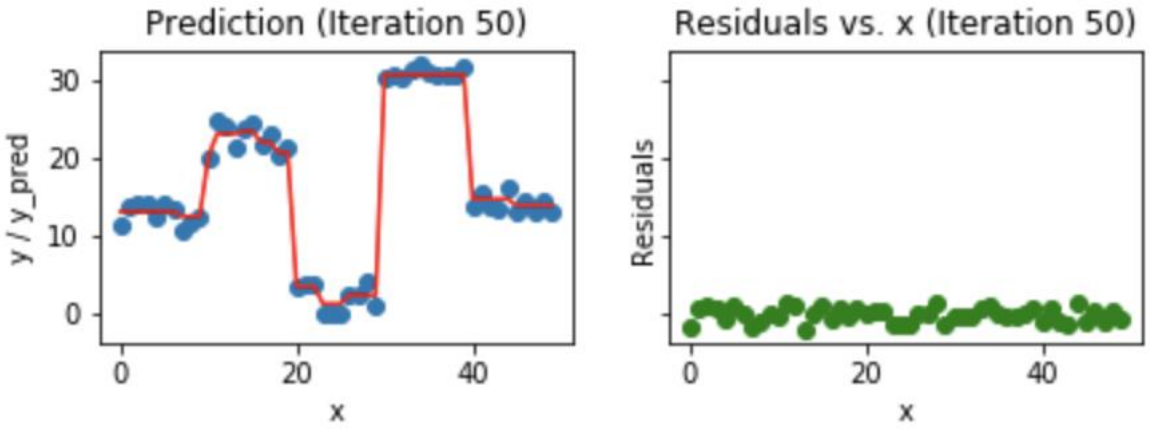


Fig 7. Visualization of gradient boosting prediction (iteration 50th)

We see that even after 50th iteration, residuals vs. x plot look similar to what we see at 20th iteration. But the model is becoming more complex and predictions are overfitting on the training data and are trying to learn each training data. So, it would have been better to stop at 20th iteration.